

Sincronización incremental del catálogo de productos

Endpoint **product_quants** - WMS OnPoint - Propuesta técnica para backend (Odoo) y app móvil - 29/09/2026

1. Contexto y diagnóstico

Firebase Performance (últimos 7 días, v1.7.1) muestra que las peticiones a **sumatec.360software.com.co** tienen un p90 de **11.29 s** (+246%) y un tamaño de respuesta p90 de **34.66 MB** (+560%). Midiendo en el dispositivo cada endpoint al iniciar sesión:

Endpoint	Tamaño (JSON)	Tiempo	Registros
GET /api/terceros	44.53 MB	78.4 s	322.109 terceros
GET /op/api/product_quants	44.11 MB	51.2 s	73.914 productos + 151.000 barcodes
GET /api/ubicaciones	3.81 MB	4.0 s	19.422 ubicaciones
Resto de endpoints	< 0.01 MB	< 2 s	-

- El servidor **ya comprime con gzip**: por la red viajan pocos MB. El cuello de botella es **Odoo generando la respuesta** (~51 s), no la descarga.
- El catálogo se descarga **completo en cada inicio de sesión**. La sesión expira cada 4 h y al cerrar sesión la app borra el catálogo local, por lo que cada relogin vuelve a pedir los 74 mil registros.
- **product_quants devuelve lo mismo para todos los usuarios del cliente** (no filtra por usuario, bodega ni permisos), así que el catálogo puede conservarse entre sesiones y usuarios de la misma empresa.

2. Solución propuesta

Conservar el catálogo local por **empresa** (no por usuario) y, en los siguientes inicios de sesión, pedir al backend **solo lo que cambió** desde la última sincronización.

Flujo

Paso	Condición	Acción
1	Login de cualquier usuario del cliente	Evaluar estado del catálogo local
2	Catálogo local es de otra empresa (o no existe)	Borrar catálogo y descarga COMPLETA
3	Misma empresa, sin lastSync o lastSync > 7 días	Descarga COMPLETA
4	Misma empresa y lastSync < 7 días	Descarga INCREMENTAL con <i>since = lastSync</i>
5	Servidor responde <i>full: true</i>	Tratar como descarga completa
6	Sincronización terminada con éxito	Guardar <i>lastSync = server_time</i> e ir al Home
7	Logout / sesión expirada (4 h)	Borrar datos de sesión, NO el catálogo ni lastSync

3. Propuesta para el backend (Odoo)

Mismo endpoint **GET /op/api/product_quants** con un parámetro opcional. Sin *since* el comportamiento es idéntico al actual (compatibilidad con versiones instaladas de la app).

Request

```
{"params": {"since": "2026-09-29 15:46:00"}} // UTC. Sin "since" -> todo
```

Response

```
{"result": {  
  "server_time": "2026-09-29 16:02:11", // UTC, hora del SERVIDOR  
  "full": false, // true = forzar recarga completa  
  "data": [ ...mismo formato actual por registro... ],  
  "deleted_product_ids": [123, 456]  
}}
```

Reglas

#	Regla
1	Un producto "cambió" si tiene fecha > <i>since</i> en cualquiera de: product.product / product.template (write_date), stock.quant (write_date), stock.move.line en estado done (date) - cubre quants eliminados al quedar en 0 -, barcodes adicionales y product.packaging (write_date), stock.lot (write_date, vencimientos).
2	Por cada producto cambiado, devolver TODAS sus filas actuales (todos sus quants ubicación/lote y barcodes), no solo la fila modificada. La app reemplaza el producto completo.
3	deleted_product_ids : productos archivados (active=False) o eliminados desde <i>since</i> .
4	server_time se toma al INICIO de la consulta (no al final) para no perder cambios ocurridos mientras se arma la respuesta. Nunca usar la hora del dispositivo.
5	full: true permite al backend forzar recarga completa (importación masiva, since muy antiguo, etc.).
6	El resultado es igual para todos los usuarios del cliente (sin filtro por usuario).
7	Optimizar la consulta completa: una sola query SQL o search_read con solo los campos necesarios, evitando recorrer registros uno a uno con el ORM (hoy ~51 s).

4. Cambios en la app (Flutter)

#	Cambio	Ubicación
1	No borrar el catálogo al cerrar sesión: sacar <code>deleInventario()</code> de <code>deleteBDCloseSession()</code> y verificar que <code>deleAllBarcodes()</code> no toque <code>BarcodesInventarioTable</code> .	<code>database.dart</code> (<code>deleteBDCloseSession</code>)
2	Prefs que sobreviven al logout: <code>catalogEnterprise</code> (URL + BD) y <code>catalogLastSync</code> . Excluir las de <code>PrefUtils.clearPrefs()</code> .	<code>PrefUtils</code>
3	Cambio de empresa: si la empresa elegida difiere de <code>catalogEnterprise</code> , ejecutar <code>deleInventario()</code> y limpiar <code>catalogLastSync</code> .	Flujo enterprise / <code>PostLoginCoordinator</code>
4	Enviar since cuando exista <code>lastSync</code> válido (misma empresa, < 7 días). <code>getInventario</code> hoy envía params vacíos: permitir pasar params.	<code>inventario_remote_data_source.dart</code> , <code>api_request_service.dart</code> (<code>getInventario</code>)
5	Parsear respuesta nueva: <code>server_time</code> , <code>full</code> , <code>deleted_product_ids</code> . Si no viene <code>server_time</code> = backend antiguo -> tratar como completa.	<code>_parseProductosIsolate</code>
6	Aplicar resultado. Completa: DROP + insert (actual). Incremental, en una sola transacción: DELETE de filas y barcodes de los <code>product_id</code> recibidos + eliminados, luego INSERT. Índices únicos existentes: (<code>product_id</code> , <code>lot_id</code> , <code>location_id</code>) y (<code>id_product</code> , <code>barcode</code>).	<code>inventario_repository_impl.dart</code> + local datasource
7	Guardar catalogLastSync = server_time solo si todo terminó OK; si falla a medias se conserva el <code>since</code> anterior y se reintenta.	Repositorio
8	(Opcional) Sincronizar en segundo plano si ya hay catálogo local, sin bloquear el Home.	<code>inventario_bloc.dart</code>

5. Orden de despliegue

- **La app puede salir primero.** Mientras el backend no devuelva `server_time`, funciona con descarga completa como hoy, pero ya no borra el catálogo al cerrar sesión.
- Cuando el backend soporte *since*, los siguientes logins pasan a incrementales automáticamente, sin nueva versión de la app.
- Red de seguridad: descarga completa si `lastSync` supera 7 días o el servidor envía `full: true`.

6. Resultado esperado

- Primer login en la PDA por empresa: igual que hoy (descarga completa).
- Relogins (sesión expirada, cambio de usuario del mismo cliente): de ~51 s y ~44 MB a pocos segundos y solo los productos modificados.
- Menor carga en el servidor Odoo y menor riesgo de alcanzar el timeout de 100 s a medida que crece el catálogo.